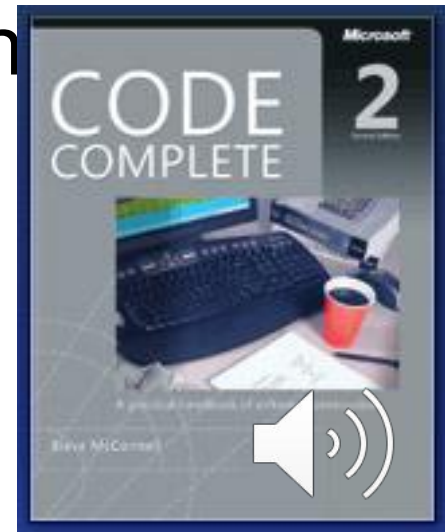


Refactoring



Software Craftsmanship

- Programming is neither fully an art nor fully a science. As it's typically practiced, it's a “craft” that's somewhere between art and science. At its best, it's an engineering discipline that arises from the synergistic fusion of art and science (McConnell 2004).



Software Craftsmanship for Maintenance

- 编程工作的一小部分是写让机器能理解的程序，大部分工作是写让人(自己和他人)能看懂的程序
 - Refactoring
 - Layout and Style
 - Self-Documenting Code



Kinds of Software Evolution

- Software evolution is like biological evolution in that some mutations are beneficial and many mutations are not.
 - The key distinction between kinds of software evolution is whether the program's quality improves or degrades under modification.
 - A second distinction in the kinds of software evolution is the one between changes made during construction and those made during maintenance.



Philosophy of Software Evolution

- A common weakness in programmers' approaches to software evolution is that it goes on as an un-self-conscious process.
 - If you recognize that evolution during development is an inevitable and important phenomenon and plan for it, you can use it to your advantage.
 - Evolution should improve the internal quality of the program.



Introduction to Refactoring

- “A change made to the internal structure of the software to make it easier to understand and cheaper to modify without changing its observable behavior” (Fowler 1999).
- Factoring:
 - decomposing a program into its constituent parts as much as possible (Yourdon and Constantine 1979).



Reasons to Refactor

- Sometimes code degenerates under maintenance, here are some warning signs sometimes called “smells” (Fowler 1999) that indicate where refactoring are needed.



Fowler Sample

- Demo



Reasons to Refactor

- Code is duplicated
 - Duplicated code almost always represents a failure to fully factor the design.
 - Duplicate code sets you up to make parallel modifications—whenever you have to make changes in one place, you have to make parallel changes in another place.
 - “Copy and Paste is a design error” (McConnell 1998b).

Reasons to Refactor

- A routine is too long
 - In object-oriented programming, routines longer than a screen are rarely needed.
 - One way to improve a system is to increase its modularity—increase the number of well-defined, well-named routines that do one thing and do it well.
 - When changes lead you to revisit a section of code, take the opportunity to check the modularity of the routines in that section.

Reasons to Refactor

- A class interface does not provide a consistent level of abstraction
 - Interface can lose their original consistency. Class interfaces tend to morph over time as a result of modifications that are made in the heat of the moment and that favor expediency to interface integrity.

Reasons to Refactor

- A loop is too long or too deeply nested
 - Loop tend to be good candidates for being converted into routines, which helps to better factor the code and to reduce the complexity of the loop.

```
if (condition1) {  
    /* ... */  
    if (condition2) {  
        /* ... */  
        for(int i = 0; i < 10; i++) {  
            /* ... */  
            if (condition4) {  
                if (condition5) {  
                    /* ... */  
                }  
                return;  
            }  
        }  
    }  
}
```

Reasons to Refactor

- Data members are public
 - Public data members are, in my view, always a bad idea. They blur the line between interface and implementation. They inherently violate encapsulation and limit future flexibility. Strongly consider hiding public data members behind access routines.

Reasons to Refactor

- Data members are public

```
class HolidayManager{  
    public char[365] days;  
}
```

```
class HolidayManager{  
    private char[365] days;  
    public boolean isHoliday(Date date){  
        .....  
    }  
}
```

Reasons to Refactor

- Changes require parallel modifications to multiple classes/locations
 - When you find yourself routinely making changes to the same set of classes, that suggests the code in those classes could be rearranged so that changes affect only one class.

Reasons to Refactor

- A primitive data type is overloaded
 - Primitive data types can be used to represent an infinite number of real-world entities.
 - If your program uses a primitive data type like an integer to represent a common entity such as money, consider creating a simple Money class so that the compiler can perform type checking on Money variables, so that you can add safety checks on the values assigned to money, and so on.
 - If both Money and Temperature are integers, the compiler won't warn you about erroneous assignments like
bankBalance = recordLowTemperature.

Reasons to Refactor

- A class doesn't do very much
 - Sometimes the result of refactoring code is that an old class doesn't have much to do. If a class doesn't seem to be carrying its weight, ask if you should assign all of that class's responsibilities to other classes and eliminate the class altogether.

Reasons to Refactor

- Comments are used to explain difficult code
 - Comments have an important role to play, but they should not be used as a crutch to explain bad code.
 - “Don’t document bad code—rewrite it” (Kernighan and Plauger 1978).

Reasons to Refactor

- Global variables are used
 - When you revisit a section of code that uses global variables, take time to reexamine them. You might have thought of a way to avoid using global variables since the last time you visited that part of the code.

Reasons to Refactor

- A program contains code that seems like it might be needed someday
 - Programmers are notoriously bad at guessing what functionality might be needed someday. “Designing ahead” is subject to numerous predictable problems.
 - Experts agree that the best way to prepare for future requirements is not to write speculative code; it’s to make the currently required code as clear and straightforward as possible so that future programmers will know what it does and does not do, and can make their changes accordingly

Reasons to Refactor

- Related data items that are used together are not organized into classes
 - If you find yourself repeatedly manipulating the same set of data items, you should ask whether those manipulations should be combined into a class of their own.

Reasons to Refactor

- A class has poor cohesion
 - If you find a class that takes ownership for a lot of unrelated responsibilities, that class should be broken up into multiple classes, each of which has responsibility for a cohesive set of responsibilities.

Reasons to Refactor

- A routine uses more features of another class than of its own class
 - This suggests that the routine should be moved into the other class and then invoked by its old class.

Reasons to Refactor

- A parameter list has too many parameters
 - Well-factored programs tend to have many small, well-defined routines that don't need large parameter lists. A long parameter list is a warning that the abstraction of the routine interface has not been well thought out.
 - long parameter lists are hard to understand.
 - if you don't have something you need, you can always ask another object to get it for you.


```

public EntryDataOutputListOutputDAOObject[] selectOutputDataEntryData(
    SepOutputFormSetupDAOObject[] _conditionDAOObject,
    String[] _strCondition1,
    String[] _strCondition2,
    String[] _strCondition3,
    String[] _strCondition4,
    String[] _strSearchOperatorDiv,
    SepOutputFormSetupDAOObject[] _outputDAOObject,
    SepOutputFormSetupDAOObject[] _tableDAOObject,
    SepOutputFormSetupDAOObject[] _sortDAOObject,
    String _strAccessAuthorityDiv,
    String _strUserId,
    ArrayList _listOrganization,
    ArrayList _aryWfStatus,
    ArrayList _aryDataEntryDiv,
    EntryDataOutputListOutputControllerObject _controllerObject
) throws ScmSystemException {
    .....
}

```

Reasons to Refactor

- Changes within a class tend to be compartmentalized
 - Sometimes a class has two or more distinct responsibilities. When that happens you find yourself changing either one part of the class or another part of the class—but few changes affect both parts of the class.
 - That's a sign that the class should be cleaved into multiple classes along the lines of the separate responsibilities.

Reasons to Refactor

- Inheritance hierarchies have to be modified in parallel
 - Finding yourself making a subclass of one class every time you make a subclass of another class is a special kind of parallel modification.

Reasons to Refactor

- Case statements have to be modified in parallel
 - Case statements are not inherently bad, but if you find yourself making parallel modifications to similar case statements in multiple parts of the program, you should ask whether inheritance might be a better approach.

Reasons to Refactor

- One class is overly intimate with another
 - Encapsulation (information hiding) is probably the strongest tool you have to make your program intellectually manageable and to minimize ripple effects of code changes.
 - Anytime you see one class that knows more about another class than it should (including derived classes knowing too much about their parents), change to the side of stronger encapsulation rather than weaker.

Reasons to Refactor

- A routine has a poor name
 - If a routine has a poor name, change the name of the routine where it's defined, change the name in all places it's called, and then recompile. As hard as it might be to do this now, it will be even harder later, so do it as soon as you notice it's a problem.

Reasons to Refactor

- A subclass uses only a small percentage of its parents' routines
 - Typically this indicates that that subclass has been created because a parent class happened to contain the routines it needed, not because the subclass is logically a descendent of the superclass.

Reasons to Refactor

- A routine uses setup code before a routine call or takedown code after a

```
rol WithdrawalTransaction withdrawal;  
withdrawal.SetCustomerId( customerId );  
withdrawal.SetBalance( balance );  
withdrawal.SetWithdrawalAmount( withdrawalAmount );  
withdrawal.SetWithdrawalDate( withdrawalDate );  
  
ProcessWithdrawal( withdrawal );  
  
customerId = withdrawal.GetCustomerId();  
balance = withdrawal.GetBalance();  
withdrawalAmount = withdrawal.GetWithdrawalAmount();  
withdrawalDate = withdrawal.GetWithdrawalDate();
```


Reasons to Refactor

- A routine uses setup code before a routine call or takedown code after a routine call

```
withdrawal = new WithdrawalTransaction( customerId, balance,  
    withdrawalAmount, withdrawalDate );  
ProcessWithdrawal( withdrawal );
```

Reasons **Not** To Refactor

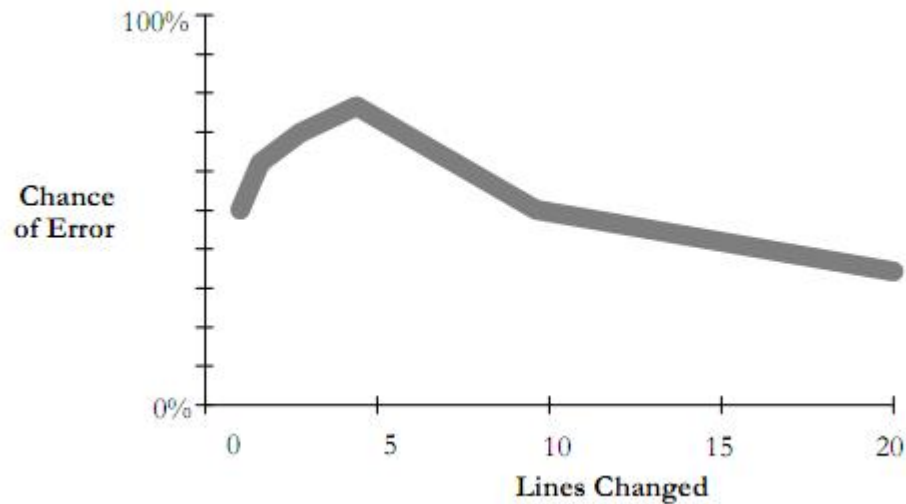
- “refactoring” should not be used to refer to fixing defects, adding functionality, modifying the design—essentially as a synonym for making any change to the code whatsoever.
 - Don’t use refactoring as a cover for code and fix
 - Avoid refactoring instead of rewriting

Refactoring Safely

- Save the code you start with
- Keep refactorings small
- Do refactorings one at a time
- Make a list of steps you intend to take
- Make frequent checkpoints
- Use your compiler warnings
- Retest
- Add test cases
- Review the changes

Refactoring Safely

- Refactoring Safely
 - Small changes tend to be more error prone than larger changes (Weinberg 1983).



Refactoring Strategies

- Refactor when you add a routine
- Refactor when you add a class
- Refactor when you fix a defect
- Target error-prone modules
- Target high complexity modules
- In a maintenance environment, improve the parts you touch
- Define an interface between clean code and ugly code, and then move code across the interface